
Toward Unified Feature Interaction and Sequence Modeling: Improvements on the PCVRHyFormer Baseline for CTR Prediction

Zijian Xu Zesong Qiu Qianqian Yang
Sauhon Wong Bingchen Zhang Xingzhou Xu
The Hong Kong Polytechnic University

Project video: <https://youtu.be/-JgE9-YpR0I>

Abstract

Click-through rate (CTR) prediction in industrial recommender systems requires jointly modeling static user/item attributes and temporal behavioral sequences, yet the two capability families — feature interaction models and sequential recommendation models — have largely evolved in isolation. We study this fusion problem on the TAAC 2026 Academic Track dataset ($\sim 1\text{M}$ anonymized Tencent ad impressions, 120 columns, 4 behavioral domains). Starting from the official PCVRHyFormer baseline, we first optimize its training configuration (Focal Loss, LongerEncoder, RoPE, ItemFeatureInteraction) to reach AUC 0.7952. We then implement PCVRInterFormer, an InterFormer-inspired architecture that replaces query-mediated token mixing with PersonalizedFeedForward modulation and cross-domain summary attention. Without hyperparameter tuning, PCVRInterFormer achieves AUC 0.8067 (+0.0115 over the optimized baseline) at half the model dimension. Ablation experiments further reveal that naive training modifications — oversized batches and aggressive embedding reinitialization — can negate architectural gains, highlighting the importance of disciplined optimization under low positive sample rates.

1 Introduction

Modern industrial recommender systems have evolved along two largely independent trajectories. *Feature interaction models* — from Factorization Machines [FM] through DeepFM [DeepFM], DCN [DCN], and Wukong [Wukong] — capture rich cross-feature dependencies over static user and item attributes. *Sequential recommendation models* — from GRU4Rec [GRU4Rec] through DIN [DIN], SASRec [SASRec], and HSTU [HSTU] — exploit the temporal structure of user behavior histories. Despite a decade of parallel progress on both fronts, unifying these two paradigms within a single architecture remains an open and practically important problem.

This separation imposes four concrete costs on production systems. (i) *Shallow cross-paradigm interaction*: most deployed pipelines combine the two branches only at the final prediction layer, discarding rich cross-modal signals that could emerge from earlier fusion. (ii) *Inconsistent optimization objectives*: sequential branches often rely on contrastive or auxiliary losses while interaction branches optimize binary cross-entropy, complicating joint training and convergence. (iii) *Limited scalability*: unlike large language models, which exhibit clear scaling laws, recommender architectures plateau quickly when model capacity is increased — the field lacks a reliable recipe for translating additional compute into proportional performance gains. (iv) *Engineering complexity*: maintaining two het-

erogeneous code paths with separate optimizers, serving stacks, and monitoring pipelines inflates deployment and iteration costs.

The TAAC 2026 Academic Track provides a controlled testbed for investigating these challenges. The competition releases approximately one million fully anonymized impression records from Tencent advertising logs, structured into 120 columns spanning static user/item features and four heterogeneous behavioral domains (Section 3). An official baseline, *PCVRHyFormer* — a HyFormer-based hybrid Transformer that fuses static and sequential information through query-mediated token mixing — is provided alongside a strict inference-latency budget that rules out naive parameter scaling.

Starting from this baseline, we make the following contributions:

- We systematically optimize the PCVRHyFormer baseline by doubling the model dimension ($d=64 \rightarrow 128$), switching to Focal Loss, adopting the LongerEncoder with RoPE, and introducing an *ItemFeatureInteraction* module, achieving AUC 0.7952.
- We implement *PCVRInterFormer*, an alternative architecture inspired by InterFormer [InterFormer] that replaces the query-token bottleneck with direct NS-sequence modulation via *PersonalizedFeedForward* and cross-domain summary attention. Without any hyperparameter tuning, PCVRInterFormer reaches AUC 0.8067, outperforming the optimized HyFormer by +0.0115 at half the model dimension.
- We conduct detailed ablation and failure analysis, showing that naive training modifications (oversized batches, aggressive embedding reinitialization) can negate architectural gains, and identify the design principles that lead to stable improvements despite the low positive rate of the dataset.

2 Related Work

2.1 Feature Interaction Models

Feature interaction models aim to capture cross-feature dependencies among static categorical and dense attributes. Factorization Machines [FM] represent pairwise interactions through low-rank factorization. DeepFM [DeepFM] and DCN [DCN] extend this idea by combining explicit cross-products with deep MLPs, while DCNv2 [DCNv2] introduces a mixture-of-experts formulation for more expressive high-order crosses. AutoInt [AutoInt] applies multi-head self-attention directly over feature embeddings, and FinalMLP [FinalMLP] demonstrates that a carefully designed two-stream MLP can match or exceed attention-based methods. More recently, Wukong [Wukong] proposes a scalable factorization framework for industrial-scale feature spaces. None of these architectures, however, model temporal user behavior; they treat each impression as an independent tabular row.

2.2 Sequential Recommendation Models

Sequential recommendation models exploit the temporal structure of user behavior. GRU4Rec [GRU4Rec] initiated the line with recurrent encoders for session-based recommendation. DIN [DIN] and DIEN [DIEN] introduced target-aware attention to weight historical actions by relevance to the candidate item, significantly improving CTR prediction in industrial settings. Transformer-based encoders — SASRec [SASRec], BERT4Rec [BERT4Rec], and BST [BST] — pushed the field toward longer contexts and richer attention patterns. Recent work such as TWIN [TWIN] and HSTU [HSTU] explores extreme sequence lengths under industrial latency budgets. These models rely primarily on sequential signals and integrate static features only superficially, typically through late concatenation.

2.3 Unified Architectures

Bridging feature interaction and sequential modeling has emerged as an active research direction. We surveyed three recent designs that directly informed the present work.

HyFormer. HyFormer [HyFormer] is a unified hybrid Transformer that tightly integrates long-sequence modeling and feature interaction within a single backbone. It frames the modeling task as alternating *Query Decoding* (expanding NS features into global tokens and decoding long sequences

via cross-attention) and *Query Boosting* (cross-query and cross-sequence interaction via RankMixer-style token mixing). HyFormer is the architectural basis of the official PCVRHyFormer baseline released for this competition; its concrete instantiation is described in detail in Section 4.1.

InterFormer. InterFormer [InterFormer] addresses two limitations of prior fusion designs: *insufficient inter-mode interaction* caused by unidirectional information flow, and *aggressive aggregation* caused by early summarization. It introduces a three-module decomposition — an interaction arch for NS features, a sequence arch for behavior histories, and a cross arch that mediates between the two via personalized feed-forward modulation (FiLM-style conditioning of sequence tokens by NS context) and cross-domain summary attention. Compared to HyFormer, InterFormer removes the explicit query-token bottleneck and lets static features modulate every position of every sequence.

OneTrans. OneTrans [OneTrans] pursues the unification argument to its logical extreme: a single Transformer over a unified token stream containing both sequential and non-sequential features, with no architectural boundary between the two. A unified tokenizer maps all features into one sequence, and stacked OneTrans Pyramid Blocks progressively reduce the token length while sharing parameters across similar sequence positions. Cross-request KV-caching is leveraged for substantial inference speed-ups.

Architectural choice for this work. Among these three, we selected InterFormer as our principal alternative to the official HyFormer-based baseline. Two considerations guided this decision. First, the academic-track dataset ($\sim 1\text{M}$ samples, four heterogeneous behavioral domains) is comparatively small; a fully unified single-stream architecture such as OneTrans is reported to benefit from industrial-scale data and aggressive caching infrastructure not available in this setting. Second, InterFormer’s PersonalizedFeedForward provides an explicit inductive bias for NS-to-sequence conditioning, well-aligned with the heterogeneity of the schema (Section 3). We thus treat HyFormer as the baseline and InterFormer as the comparison architecture; OneTrans is left for future investigation under larger-scale training conditions.

3 Problem Formulation and Dataset

3.1 Task Formulation

We formulate click-through rate (CTR) prediction as a binary classification task. Given a user u , a candidate item i , and the user’s historical behavior sequences across $D = 4$ heterogeneous domains $S = \{S_a, S_b, S_c, S_d\}$, the model outputs a click probability

$$\hat{y} = f_{\theta}(u, i, S) \in [0, 1], \quad (1)$$

where f_{θ} denotes the parameterized model with learnable parameters θ . The model is trained by minimizing the standard binary cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N [y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n)], \quad (2)$$

where $y \in \{0, 1\}$ is the ground-truth label indicating whether the impression resulted in a click, and N is the number of training samples. At inference time, predicted probabilities are used to rank candidate items; the system is evaluated by the area under the ROC curve (AUC), subject to the official latency constraint imposed by the competition organizers.

3.2 Dataset Overview

We use the *TAAC 2026 Academic Track* dataset, which consists of approximately one million impression records derived from real-world Tencent advertising logs. Each record corresponds to a single (user, item, timestamp) impression, augmented with the user’s preceding behavior across multiple domains.

The dataset is *fully anonymized*: all sparse features are mapped to integer IDs, all dense features are presented as fixed-length float vectors, and no raw content (text, images, URLs) or personally identifiable information is preserved. As a consequence, *pre-trained language or vision models cannot*

be leveraged for feature extraction — methodological contributions must come from architecture design rather than external knowledge transfer.

3.3 Feature Schema

Each record is stored in a *flat schema of 120 columns*, partitioned into six categories (Table 1): five ID/label columns, 46 user integer features (35 scalar + 11 multi-value), 10 user dense features, 14 item integer features, zero item dense features, and 45 domain sequence features distributed across four behavioral domains.

A distinctive feature of the schema is the *aligned (key, value) structure*: eight of the ten user-dense columns are *positionally aligned* with their integer counterparts, encoding per-element behavioral statistics (e.g., dwell time, interaction score) for the corresponding entity IDs.

Table 1: Schema of the TAAC 2026 Academic Track dataset (120 columns).

Category	Count	Type	Description
ID & label	5	int64 / int32	Identifiers, click label, timestamp
User integer	46	int64 / list	35 scalar + 11 multi-value
User dense	10	list(float)	2 pre-trained + 8 aligned
Item integer	14	int64 / list	13 scalar + 1 multi-value
Item dense	0	—	(none)
Domain sequences	45	list(int64)	4 domains: A (9), B (14), C (12), D (10)
Total	120		

3.4 Behavioral Domains

The 45 sequence columns are partitioned into four *heterogeneous behavioral domains*, denoted A through D , with column counts 9, 14, 12, and 10 respectively. While the precise business semantics of each domain are not disclosed by the dataset provider, the domains differ in average sequence length and likely correspond to distinct interaction modalities (e.g., feed impressions, search interactions, video views, application usage). To preserve this heterogeneity, our model treats each domain as a separate sequence stream, and any cross-domain reasoning is deferred to the fusion stage rather than performed by collapsing all actions onto a single timeline.

Within each domain, multiple columns are *positionally aligned*: the i -th element across all columns of domain d describes the i -th historical action through complementary attributes such as item identifier, category, and timestamp. Continuous timestamps are not consumed directly; instead, inter-action time deltas are discretized into 65 logarithmically-spaced buckets ranging from 5 seconds to one year, with bucket 0 reserved for padding. Each bucket maps to a learnable embedding, providing the model with a smooth temporal prior that distinguishes recent from distant behavior without exposing raw Unix time. Following the official baseline configuration, sequences are truncated to a maximum length of 256 for domains A and B , and 512 for the longer domains C and D .

4 Methodology

4.1 Baseline Architecture: PCVRHyFormer

4.1.1 Overview

This section describes the official baseline released by the competition organizers, denoted *PCVRHyFormer*. We use this architecture without modification as our starting point; the contributions of this work, presented in Section 4.2, are layered on top of it. Our goal here is to make the baseline self-contained for the reader, so that subsequent comparisons in Section 5 can be interpreted unambiguously.

PCVRHyFormer is a *unified hybrid Transformer* that processes static user/item attributes and temporal behavior sequences within a single architecture, rather than fusing them only at the output layer. The model maintains three semantically distinct sets of tokens — *non-sequential (NS) tokens* derived

from static features, *sequence tokens* derived from per-domain behavioral histories, and a small set of learnable *query (Q) tokens* that mediate cross-modal information transfer. These tokens are processed jointly through N stacked *MultiSeqHyFormerBlocks*, after which the final query tokens are pooled and passed to a classifier head producing the click probability $\hat{y}$.

4.1.2 Embedding Layer

The embedding layer maps raw integer IDs and dense values into a shared d -dimensional vector space, producing the inputs to all subsequent token streams. Three distinct embedding strategies are employed, reflecting the heterogeneity of the 120-column schema.

Sparse feature embeddings. Each integer feature with vocabulary size V is assigned an embedding table $E \in \mathbb{R}^{(V+1) \times d_{\text{emb}}}$, where index 0 is reserved for padding and Xavier-initialized to zero. Multi-value features are mean-pooled across their non-zero entries before being passed downstream. To accommodate features with extremely large cardinalities — for which a full embedding table would exceed practical memory budgets — the baseline applies an *embedding skip threshold*: any feature whose vocabulary size exceeds $V_{\text{skip}} = 10^6$ is replaced with a zero vector and effectively dropped from the model. This is a deliberate engineering trade-off that bounds the parameter footprint at the cost of discarding a small number of long-tail signals.

Dense feature projections. The 10 user-dense columns are concatenated into a single vector and passed through a two-layer block (linear projection + LayerNorm + SiLU) that yields a single d -dimensional NS token. The (key, value)-aligned columns described in Section 3.3 are *not* given special treatment in the baseline: their integer counterparts are embedded independently as sparse features, and their dense counterparts are fused into the same flattened user-dense vector, leaving the positional alignment between the two unexploited at the input stage.

Time-bucket embeddings. Continuous inter-action time deltas are quantized into 65 logarithmically-spaced buckets (Section 3.4) and mapped through a learnable embedding table $E_t \in \mathbb{R}^{65 \times d}$, with bucket 0 reserved for padding. Time-bucket vectors are added element-wise to their corresponding sequence tokens, providing a learned temporal prior without exposing raw Unix timestamps to the network.

4.1.3 Token Types

We now describe how the three token streams are constructed from the embedded features. Throughout, let D denote the model dimension ($D = 64$ in the default configuration).

Non-sequential (NS) tokens. NS tokens encode static user and item attributes that do not depend on temporal context. The baseline adopts the *RankMixer*-style tokenizer [RankMixer]: all sparse embeddings within a feature group are concatenated into a single vector, equally split into T_{ns} chunks, and each chunk is independently projected to $\mathbb{R}^D$ via a linear layer with LayerNorm. This produces $T_{\text{ns}}^u = 5$ user NS tokens and $T_{\text{ns}}^i = 2$ item NS tokens. The user-dense vector contributes one additional NS token (Section 4.1.2), giving $M = T_{\text{ns}}^u + 1 + T_{\text{ns}}^i = 8$ NS tokens in total.

Sequence tokens. Each behavioral domain $d \in \{A, B, C, D\}$ is tokenized independently. For position ℓ in domain d , the embeddings of all aligned side-information columns (item ID, category, etc.) are concatenated and projected to $\mathbb{R}^D$, then summed with the corresponding time-bucket embedding. The result is a domain-specific sequence tensor of shape (L_d, D) , where $L_d \in \{256, 256, 512, 512\}$ following the truncation policy of Section 3.4. Crucially, the four domains do *not* share a unified embedding stack: each domain maintains its own embedding tables, so no cross-domain ID identification is performed at this stage.

Query (Q) tokens. Q tokens act as learnable *read-out probes* that extract task-relevant signals from the sequences. For each domain, the baseline first constructs a *global context vector* by concatenating the flattened NS tokens with the mean-pooled sequence tokens of that domain, and then transforms this context through N_q independent feed-forward networks to produce N_q query tokens (with $N_q = 2$ in the default configuration). Q tokens are thus *initialized from* both static and behavioral

information, but their final values emerge only after multiple rounds of cross-attention and mixing within the block stack (Section 4.1.4).

4.1.4 MultiSeqHyFormerBlock

The core computation of PCVRHyFormer takes place inside a stack of N identical *MultiSeqHyFormerBlocks* (default $N = 2$). Each block consumes the current state of all three token streams and produces refined versions of the same shape, allowing the architecture to be cleanly stacked. Within a single block, four operations are executed in sequence.

(i) Sequence evolution. Each domain d first refines its own sequence tokens through an independent Transformer encoder, equipped with rotary position embeddings (RoPE) [RoFormer] and a padding mask. Self-attention is restricted to within-domain tokens, so the four domains do not exchange information at this stage. Formally, for each d ,

$$S_d \leftarrow \text{SeqEncoder}_d(S_d, \text{mask}_d), \quad (3)$$

where $S_d \in \mathbb{R}^{L_d \times D}$ denotes the sequence tokens of domain d .

(ii) Query decoding. The N_q query tokens of each domain then attend over their refined sequence tokens via cross-attention, with the queries acting as the *query* side and the sequence tokens as *key* and *value*. RoPE is applied only on the key/value side. This step lets each query token gather behaviorally relevant context from its assigned domain:

$$Q_d \leftarrow \text{CrossAttn}_d(Q_d, S_d), \quad d \in \{A, B, C, D\}. \quad (4)$$

(iii) Token fusion. All decoded query tokens and the shared NS tokens are concatenated into a single token stream of length $T = N_q \cdot |D| + M$ (with default $T = 2 \cdot 4 + 8 = 16$):

$$Z = \text{concat}(Q_A, Q_B, Q_C, Q_D, \text{NS}) \in \mathbb{R}^{T \times D}. \quad (5)$$

This concatenation is the only point at which static and behavioral information are placed in a common attention context, making it the architectural locus of *deep* static–sequential fusion.

(iv) Query boosting. The fused stream Z is finally processed by a *RankMixer*-style block that performs token mixing and a per-token feed-forward refinement [RankMixer]. Token mixing is parameter-free: each token’s D -dimensional representation is reshaped into T sub-spaces of size D/T and the token and sub-space axes are transposed, redistributing information across tokens before a shared FFN refines each token independently. A residual connection and post-LayerNorm yield the boosted output, which is finally split back into per-domain Q_d and NS slots ready to be consumed by the next block. The construction requires $D \bmod T = 0$, a constraint that, together with the chosen number of NS tokens, fixes the admissible values of D in the default configuration.

After the final block, only the per-domain query tokens are retained: they are concatenated, flattened, and projected through a linear–LayerNorm head to produce a single D -dimensional pooled vector, which is then passed to the classifier described in Section 4.1.5.

4.1.5 Output and Training

Classifier head. The pooled D -dimensional output of the block stack is passed through a lightweight classifier consisting of a linear layer, LayerNorm, SiLU activation, dropout, and a final linear projection to a single logit. A sigmoid then yields the predicted click probability $\hat{y}$.

Training configuration. The model is trained end-to-end with the binary cross-entropy loss of Equation (2). Following standard practice in industrial recommender systems, parameters are split into two groups optimized by separate routines: all embedding tables (“sparse” parameters) are updated with Adagrad, while the remaining (“dense”) parameters are updated with AdamW. This split exploits the fact that embedding gradients are sparse — only a small subset of rows is touched per step — allowing Adagrad’s per-coordinate learning rates to outperform a uniform optimizer on these parameters. The default hyperparameters are: dense learning rate 1×10^{-4} , batch size 256, dropout 0.01, $D = 64$, $N_q = 2$, $N = 2$ stacked blocks, and 4 attention heads. Early stopping is applied on the held-out validation AUC with a patience of five evaluation rounds.

4.2 Our Approach: PCVRInterFormer

Motivated by the architectural limitations of the query-mediated fusion in PCVRHyFormer — in particular, the per-domain isolation of query tokens and the rigid $D \bmod T = 0$ constraint of the RankMixer block — we explore an alternative architecture inspired by the InterFormer paradigm [InterFormer]. The resulting model, *PCVRInterFormer*, replaces the three-token-stream design with a two-stream architecture that achieves static-sequential fusion through *personalized feed-forward modulation* and *cross-domain summary attention*, eliminating the need for explicit query tokens entirely.

4.2.1 Architecture Overview

PCVRInterFormer processes the same 120-column input (Section 3) through two parallel streams: (1) non-sequential (NS) tokens, constructed identically to the baseline (Section 4.1.3), and (2) per-domain sequence tokens, also embedded as in the baseline. The key difference lies in how these streams interact. Rather than mediating fusion through query tokens, PCVRInterFormer employs N stacked *InterFormerBlocks* in which NS tokens and sequence tokens directly exchange information via two complementary mechanisms described below.

4.2.2 Cross-Domain Summary

At the start of each block, a *CrossSummary* module aggregates information from all four behavioral domains into a single summary vector. It concatenates the sequence tokens of all domains into one long stream and applies multi-head attention using two learnable *summary queries* — one acting as a CLS-style global readout, the other as a Pooling-by-Multi-head-Attention (PMA) probe. Their attended outputs are combined with the mean of each domain’s most recent token via a gated fusion:

$$\mathbf{s} = \sigma(g([\mathbf{s}_{\text{cls}}; \mathbf{s}_{\text{pma}}; \mathbf{s}_{\text{recent}}])) \cdot \frac{1}{3}(\mathbf{s}_{\text{cls}} + \mathbf{s}_{\text{pma}} + \mathbf{s}_{\text{recent}}), \quad (6)$$

where $g(\cdot)$ is a two-layer MLP and σ denotes the sigmoid function. This summary vector is appended to the NS token stream and processed jointly via self-attention, allowing NS tokens to absorb cross-domain behavioral signals.

4.2.3 Personalized Sequence Modulation

After the NS-side self-attention, a context vector is derived from the updated NS tokens (via masked mean-pooling with sigmoid gating) and used to *modulate* each domain’s sequence tokens through a *PersonalizedFeedForward* (PFFN) layer. Concretely, for sequence token $\mathbf{x}$ and context $\mathbf{c}$:

$$\text{PFFN}(\mathbf{x}, \mathbf{c}) = \text{FFN}(\mathbf{x} \odot \sigma(W_g \mathbf{c}) + W_b \mathbf{c}), \quad (7)$$

where W_g, W_b are learned projection matrices. This FiLM-style conditioning [FiLM] allows static user/item attributes to directly influence how each sequence position is processed, achieving deep fusion without an intermediate query-token bottleneck.

In the first block, a context token is additionally prepended to each domain’s sequence before self-attention, providing an explicit conditioning signal at the earliest layer.

4.2.4 Gated Output Fusion

After the final block, the NS and sequence streams are independently summarized (NS via mean-pooling, sequences via a final CrossSummary pass) and combined through a learned gate:

$$\mathbf{h} = \alpha \cdot \mathbf{h}_{\text{ns}} + (1 - \alpha) \cdot \mathbf{h}_{\text{seq}}, \quad \alpha = \sigma(W_\alpha[\mathbf{h}_{\text{ns}}; \mathbf{h}_{\text{seq}}]). \quad (8)$$

The fused vector $\mathbf{h}$ is passed to a classifier (LayerNorm $\rightarrow$ Linear $\rightarrow$ SiLU $\rightarrow$ Dropout $\rightarrow$ Linear) to produce the click logit.

4.3 Training Optimizations

Beyond the architectural change, we introduce several training-level modifications that address the class imbalance under low positive rates and training instability observed in early experiments.

Focal Loss. We replace the standard BCE loss with Focal Loss [Focal], which down-weights well-classified negatives via the modulating factor $(1 - p_t)^\gamma$:

$$\mathcal{L}_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log(p_t), \quad (9)$$

with $\alpha = 0.25$ and $\gamma = 2.0$. This prevents the overwhelming majority of easy negatives from dominating the gradient.

Regularization. The dropout rate is increased from 0.01 to 0.1 to combat overfitting on the sparse label distribution. High-cardinality ID features (vocabulary > 5000) receive double dropout on their embeddings.

Training stability. We adopt mixed-precision training with GradScaler and gradient clipping ($\|\nabla\|_{\text{max}} = 1.0$) to prevent the FP16 overflow that caused NaN divergence in earlier iterations. A linear learning-rate warmup over the first 1000 steps further stabilizes cold-start optimization.

Dual optimizer. Following the baseline, sparse embedding parameters use Adagrad ($\text{lr} = 0.01$) and all other parameters use AdamW ($\text{lr} = 10^{-4}$).

5 Experiments

5.1 Experimental Setup

All experiments use the TAAC 2026 Academic Track dataset described in Section 3. We adopt a temporal validation strategy: Row Groups are sorted by their timestamp metadata and the most recent 10% are held out for evaluation, simulating the real-world setting where models are trained on historical data and evaluated on future impressions. The primary metric is AUC of ROC (higher is better), consistent with the official competition protocol. All models are trained on a single GPU; training-time hyperparameters are reported alongside each variant in Table 2.

5.2 Main Results

Table 2 compares the official PCVRHyFormer baseline with our optimized HyFormer variant and the proposed PCVRInterFormer across multiple iterations.

Table 2: Main results on the TAAC 2026 validation set (temporal split). d : model dimension. All other hyperparameters are detailed in the text. [†]Peak validation AUC at step 2396; subsequent training exhibited overfitting (see Section 5.3).

Model	Ver.	AUC	d	Key changes vs. baseline
PCVRHyFormer	v0.2	0.7952	128	Focal, LongerEnc, RoPE, ItemInteract
PCVRInterFormer	v0.1	0.8067	64	InterFormer arch (no tuning)
PCVRInterFormer	v0.3	0.8038	64	+ Focal, batch $\uparrow$ 1024, cold restart
PCVRInterFormer	v0.5	0.8520 [†]	64	+ DHEN, DCN, GradScaler, warmup

Two findings stand out. First, the untuned PCVRInterFormer (v0.1) outperforms the heavily optimized HyFormer (v0.2) by +0.0115 AUC despite using half the model dimension ($d=64$ vs. $d=128$), suggesting that the InterFormer fusion paradigm — direct NS-sequence modulation via PersonalizedFFN and cross-domain summary attention — is inherently more effective on this dataset than the query-mediated fusion of HyFormer. Second, the v0.3 hyperparameter sweep *degraded* performance by -0.0029 AUC, a result we analyze further in Section 5.3.

5.3 Ablation and Failure Analysis

The v0.3 experiment modified three variables simultaneously relative to v0.1: (i) switching the loss from BCE to Focal Loss ($\alpha=0.1$, $\gamma=2.0$), (ii) quadrupling the batch size from 256 to 1024, and (iii) enabling per-epoch cold restart of high-cardinality embeddings (threshold = 10,000). The combined effect was a regression of -0.0029 AUC. Post-hoc diagnosis revealed three contributing factors:

1. **Under-training from large batches.** With batch size 1024 and a hard cap of 3 epochs, the total number of parameter updates was roughly one quarter of v0.1. The learning rate was not scaled to compensate, violating the linear scaling rule.
2. **Destructive cold restarts.** Reinitializing all embeddings with vocabulary $> 10,000$ after every epoch erased the representations learned for high-frequency items, preventing the model from retaining long-tail knowledge across epochs.
3. **Compound rather than additive effects.** By modifying loss, batch size, and embedding regularization simultaneously, v0.3 made it impossible to attribute the regression to any single change. Follow-up runs in v0.4 and v0.5 isolated these factors and revealed that an under-tuned Focal Loss ($\alpha = 0.1$ in v0.3 vs. $\alpha = 0.25$ in v0.5) systematically down-weighted the rare positive samples that the loss was meant to emphasize, compounding the under-training already caused by factor (i). Furthermore, the per-user GAUC metric introduced in v0.5 stays near zero throughout training even when AUC reaches its peak of 0.852, suggesting that the -0.0029 regression in v0.3 captures only a small portion of the true performance gap; the remainder is bounded by feature representation rather than by hyperparameter choices alone.

These findings informed the design of v0.5, which combines tuned Focal Loss ($\alpha=0.25$), reduced sparse learning rate ($0.05 \rightarrow 0.01$), linear warmup, and mixed-precision training with `GradScaler`. While v0.5 stabilizes training and reaches a peak AUC of 0.852, it overfits in later steps (decaying to 0.760 by step 6000) and high Precision@1% (0.92) coexists with near-zero GAUC — consistent with the analysis above and pointing to feature engineering as the most promising next direction.

5.4 Latency Considerations

All reported models satisfy the competition’s latency budget. The PCVRInterFormer v0.1 completes inference over the full validation set in 226.89 seconds; the optimized HyFormer v0.2 achieves 155.55 seconds owing to its use of the LongerEncoder’s Top-K compression. We note that the InterFormer architecture does not introduce additional latency-critical operations beyond standard multi-head attention, and its $d=64$ configuration keeps the per-sample cost well within practical limits.

6 Conclusion

We presented a systematic study of static–sequential fusion architectures for CTR prediction on the TAAC 2026 Academic Track dataset. Starting from the official PCVRHyFormer baseline, we pursued two complementary directions: (1) optimizing the baseline itself through expanded model capacity, Focal Loss, LongerEncoder with RoPE, and a novel ItemFeatureInteraction module (AUC 0.7952), and (2) implementing PCVRInterFormer, an InterFormer-inspired architecture that replaces query-mediated token mixing with direct PersonalizedFeedForward modulation and cross-domain summary attention (AUC 0.8067).

Our experiments reveal that architecture choice is the dominant factor: the untuned InterFormer surpasses the heavily optimized HyFormer by $+0.0115$ AUC at half the model dimension. Equally instructive are our negative results — naive training modifications such as oversized batches and aggressive embedding reinitialization degraded performance by -0.0029 AUC, underscoring the importance of disciplined hyperparameter tuning under low positive sample rates.

Limitations. Our study is limited to a single anonymized dataset from one advertiser ecosystem; generalization to other domains and data distributions remains to be verified. The InterFormer v0.5 variant, which incorporates DHEN and DCN interaction modules, shows promising peak performance (AUC 0.852) but exhibits overfitting in later training and a near-zero GAUC, suggesting its true potential will only be realized once the feature engineering directions discussed below are addressed.

Future work. Promising directions include feature engineering on the unexploited (key, value)-aligned dense columns (Section 4.1.2), cross-domain ID sharing analysis, and systematic scaling experiments comparing InterFormer’s scaling behavior against HyFormer.

References

- [1] Guo, H., Tang, R., Ye, Y., Li, Z. & He, X. (2017) DeepFM: A factorization-machine based neural network for CTR prediction. *IJCAI*.
- [2] Kang, W.-C. & McAuley, J. (2018) Self-attentive sequential recommendation. *ICDM*.
- [3] Zhou, G., Zhu, X., Song, C., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H. & Gai, K. (2018) Deep interest network for click-through rate prediction. *KDD*.
- [4] Huang, Y., Hong, S., Xiao, X., Jin, J., Luo, X., Wang, Z., Chai, Z., Wu, S., Zheng, Y. & Lin, J. (2026) HyFormer: Revisiting the roles of sequence modeling and feature interaction in CTR prediction. *arXiv preprint arXiv:2601.12681*.
- [5] Zeng, Z., Liu, X., Hang, M., Liu, X., Zhou, Q., Yang, C., Liu, Y., Ruan, Y., et al. (2025) InterFormer: Effective heterogeneous interaction learning for click-through rate prediction. *In Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM 2025)*, pp. 6225–6233.
- [6] Zhang, Z., Pei, H., Guo, J., Wang, T., Feng, Y., Sun, H., Liu, S. & Sun, A. (2025) OneTrans: Unified feature interaction and sequence modeling with one Transformer in industrial recommender. *arXiv preprint arXiv:2510.26104*.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction summarize the same experimental results (AUC values, ablation findings) reported in Section 5, and explicitly disclose the scope (single anonymized dataset, academic-track scale).

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 contains a dedicated *Limitations* paragraph discussing dataset scope, the v0.5 overfitting and near-zero GAUC, and the resulting bound on generalizability.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: This paper does not present new theoretical results.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

Answer: [Yes]

Justification: Section 4.3 reports all hyperparameters (Focal Loss α, γ ; dropout; learning rates; batch size; warmup), and Table 2 lists model dimensions and key changes per variant. Code is shared in the team repository.

5. Open access to data and code

Question: Does the paper provide open access to the data and code?

Answer: [Yes]

Justification: The TAAC 2026 dataset is publicly available through the official competition portal. Our implementation extends the official PCVRHyFormer codebase and is hosted in the team’s GitHub repository.

6. Experimental setting/details

Question: Does the paper specify all the training and test details?

Answer: [Yes]

Justification: Section 4.1.5 and Section 4.3 specify optimizer choices (Adagrad for sparse, AdamW for dense), all learning rates, batch size, dropout, and early-stopping criterion. Section 5 specifies the temporal 90:10 split and the AUC evaluation protocol.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined?

Answer: [No]

Justification: Due to the substantial training cost of each variant (multiple hours per training run), each configuration was trained once. All differences reported are between distinct runs without standard deviation; the AUC gap of +0.0115 between v0.1 and the optimized HyFormer baseline is large enough to be qualitatively informative without formal significance testing.

8. Experiments compute resources

Question: Does the paper provide sufficient information on compute resources?

Answer: [Yes]

Justification: All experiments were conducted on a single GPU. Validation-set inference times for each variant are reported in Section 5 (226.89s for InterFormer v0.1, 155.55s for HyFormer v0.2).

9. Code of ethics

Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics?

Answer: [\[Yes\]](#)

Justification: The research uses an officially anonymized public dataset and introduces no human subjects, surveillance applications, or dual-use risks.

10. Broader impacts

Question: Does the paper discuss both potential positive and negative societal impacts?

Answer: [\[NA\]](#)

Justification: The work studies model architecture for an officially anonymized advertising-prediction dataset and introduces no new application or deployment beyond the academic comparison setting; we identify no specific positive or negative societal impact distinct from the established CTR-prediction literature.

11. Safeguards

Question: Does the paper describe safeguards for responsible release of high-risk data or models?

Answer: [\[NA\]](#)

Justification: This paper does not release new models or datasets with high misuse risk.

12. Licenses for existing assets

Question: Are creators or original owners of assets used in the paper properly credited?

Answer: [\[Yes\]](#)

Justification: The TAAC 2026 dataset and PCVRHyFormer baseline are properly cited; their use follows the official competition terms.

13. New assets

Question: Are new assets introduced in the paper well documented?

Answer: [\[NA\]](#)

Justification: This paper does not release new public assets.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions?

Answer: [\[NA\]](#)

Justification: The paper does not involve crowdsourcing or human subjects research.

15. Institutional review board (IRB) approvals

Question: Does the paper describe potential risks incurred by study participants and IRB approvals?

Answer: [\[NA\]](#)

Justification: The paper does not involve human subjects research.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important component of the core methods?

Answer: [\[NA\]](#)

Justification: LLMs are not used as a core methodological component of this work.